

Lab 8: Table Relationships — Product Shop

ส่วนที่ 1: หลักการออกแบบ

1. SOLID Principles ที่ใช้ในการออกแบบ Entity

S — Single Responsibility Principle (SRP) แยก ProductDetail ออกจาก Product เพราะข้อมูลอย่าง warranty, weight, dimensions เป็นข้อมูลเสริมที่ไม่จำเป็นต้องดึงมาทุกครั้ง query สินค้า การแยกออกทำให้แต่ละ Entity มีหน้าที่ชัดเจน: Product รับผิดชอบข้อมูลหลักของสินค้าและตรรกะทางธุรกิจ (เช่น การคำนวณราคาส่วนลด) ส่วน ProductDetail รับผิดชอบเฉพาะข้อมูลเสริมเท่านั้น

O — Open/Closed Principle (OCP) การเพิ่ม Review เป็นตารางใหม่ที่ผูกกับ Product แบบ 1:N โดยไม่ต้องแก้ไขโครงสร้างของ Product เดิมเลย (แค่เพิ่ม field List<Review> reviews) แสดงให้เห็นว่าระบบเปิดรับการขยายฟีเจอร์ใหม่ได้ โดยไม่ต้องแก้ไข logic เดิมที่มีอยู่แล้ว

L — Liskov Substitution Principle (LSP) Repository ทั้ง 3 ตัว (ProductRepository, ProductDetailRepository, ReviewRepository) implement interface JpaRepository<T, ID> เหมือนกัน ทำให้ Service ชั้นบนสามารถเรียกใช้งานผ่าน interface เดียวกันได้โดยไม่ต้องสนใจว่า Entity เบื้องหลังต่างกัน

I — Interface Segregation Principle (ISP) แยก ProductRepository, ProductDetailRepository, ReviewRepository ออกเป็นคนละ interface แทนที่จะรวมทุก CRUD operation ไว้ใน interface เดียว ทำให้แต่ละ Repository มีขอบเขตความรับผิดชอบเฉพาะ Entity ของตัวเอง ไม่ใหญ่เทอะทะเกินไป

D — Dependency Inversion Principle (DIP) ProductService ขึ้นกับ ProductRepository ซึ่งเป็น interface ไม่ใช่ implementation class จริง (Spring Data JPA สร้าง implementation ให้อัตโนมัติตอน runtime) ทำให้ Service ไม่ผูกติดกับรายละเอียดการเข้าถึงฐานข้อมูล เปลี่ยนวิธีการ implement ได้โดยไม่กระทบ Service

2. ความแตกต่างระหว่าง 1:1 กับ 1:N

One-to-One (ProductProductDetail)

สินค้าหนึ่งรายการมีข้อมูลเสริมได้เพียงชุดเดียวเท่านั้น ความสัมพันธ์แบบนี้เหมาะกับข้อมูลที่เป็น "ส่วนขยาย" ของ Entity หลัก แต่ไม่ได้ใช้บ่อยพอที่จะเก็บรวมในตารางเดียวกัน — การแยกออกช่วยให้ query หลัก (SELECT * FROM products) เบาลง เพราะไม่ต้องดึงข้อมูลเสริมทุกครั้ง Foreign Key จะอยู่ฝั่ง Owner คือ products.detail_id

One-to-Many (Product Review)

สินค้าหนึ่งรายการมีรีวิวได้ไม่จำกัดจำนวน เหมาะกับข้อมูลแบบ "child records" ที่เพิ่มขึ้นเรื่อยๆ ตามเวลา (ลูกค้ารีวิวเพิ่มได้ตลอด) การออกแบบแบบนี้รองรับ OCP เพราะเพิ่มรีวิวใหม่ได้โดยไม่กระทบ Product เลย Foreign Key จะอยู่ฝั่ง Many คือ reviews.product_id หลักการเลือกใช้: ถ้าความสัมพันธ์มีจำนวนคงที่ที่ 1 ต่อ 1 เสมอ ใช้ 1:1 แต่ถ้าฝั่งใดฝั่งหนึ่งสามารถมีได้หลายรายการแบบไม่จำกัด ใช้ 1:N

3. Strategy Pattern สำหรับคำนวณส่วนลด

DiscountStrategy เป็น interface กลางที่กำหนด method applyDiscount(double price) มี 3 implementation คือ NoDiscountStrategy (ไม่ลด), MemberDiscountStrategy (ลด 10%), SeasonalSaleStrategy (ลด 20%)

DiscountContext ทำหน้าที่เป็นตัวกลางที่ถือ reference ไปยัง strategy ที่เลือกไว้ และมี static method resolve(discountType) ที่แปลงค่า discountType (String จากฟอร์ม เช่น "MEMBER") ให้กลายเป็น strategy object ที่เหมาะสม

ข้อดีของแบบนี้คือถ้าต้องเพิ่มโปรโมชันใหม่ในอนาคต (เช่น ส่วนลดวันเกิด) แค่สร้างคลาสใหม่ implement DiscountStrategy แล้วเพิ่มเงื่อนไขใน resolve() โดยไม่ต้องแก้ logic การคำนวณเดิมเลย — สอดคล้องกับ OCP โดยตรง

4. Execution Flow ตั้งแต่ HTTP Request → DB

ผู้ใช้อกรอกฟอร์มที่ /products/add แล้วกด submit → browser ส่ง POST /products/save พร้อม ข้อมูลฟอร์ม

ProductController.save() รับ request ผ่าน @PostMapping("/save") และ Spring MVC bind ข้อมูลจากฟอร์มเข้า object Product ผ่าน @ModelAttribute โดยอัตโนมัติ (รวมถึง nested object detail และ reviews[0]) Controller เรียก productService.saveProduct(product) — ส่งต่อให้ Service ชั้นตรรกะ ทางธุรกิจ (Controller ไม่รู้เรื่อง database เลย)

ProductService เรียก productRepository.save(product) ผ่าน interface ที่ Spring Data JPA generate implementation ให้อัตโนมัติ JPA/Hibernate แปลง object Product เป็น SQL INSERT โดยอาศัย annotation ที่ประกาศไว้ (@OneToOne cascade=ALL ทำให้ ProductDetail ถูก insert ตามไปด้วยในทรานแซกชันเดียวกัน เช่นเดียวกับ @OneToMany cascade=ALL สำหรับ Review) ข้อมูลถูกเขียนลง PostgreSQL ครบทั้ง 3 ตาราง (products, product_details, reviews) พร้อม Foreign Key เชื่อมกัน Controller ส่ง redirect:/products กลับไป → browser ยิง GET /products ใหม่ → ProductController.list() เรียก productService.getAllProducts() → ดึงข้อมูลกลับมาแสดงในหน้า list.html

ส่วนที่ 2: Code + คำอธิบาย

Entity: Product.java
java
@Entity
@Table(name = "products")
public class Product {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
...
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "detail_id", referencedColumnName = "id")
private ProductDetail detail;
@OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Review> reviews = new ArrayList<>();
}
@Entity และ @Table(name = "products"): บอก JPA ว่าคลาสนี้คือตารางในฐานข้อมูล ชื่อตารางคือ products

@Id @GeneratedValue(strategy = GenerationType.IDENTITY): กำหนด Primary Key ให้ database เป็นผู้สร้างค่าอัตโนมัติ (auto-increment) ตามที่ PostgreSQL รองรับ

@OneToOne(cascade = CascadeType.ALL): Product เป็นฝั่ง Owner ของความสัมพันธ์ 1:1 การตั้ง cascade = ALL ทำให้เมื่อ save/delete Product การเปลี่ยนแปลงจะกระจายไปถึง ProductDetail ที่ผูกอยู่ด้วยอัตโนมัติ

@JoinColumn(name = "detail_id", ...): กำหนดชื่อคอลัมน์ Foreign Key ที่จะถูกสร้างในตาราง products ขึ้นไปยัง id ของ product_details

@OneToMany(mappedBy = "product", ...): Product เป็นฝั่ง "1" ของความสัมพันธ์ 1:N คำว่า mappedBy = "product" บอกว่าความสัมพันธ์นี้ถูกจัดการ (owning side) อยู่ที่ field ชื่อ product ใน class Review ไม่ใช่ที่นี่ — ทำให้ Product ไม่ต้องสร้างคอลัมน์ FK ของตัวเอง

orphanRemoval = true: ถ้ามลบ Review ออกจาก List แล้ว save จะลบ record ออกจากฐานข้อมูลจริงด้วย ไม่ปล่อยเป็น record ลอย

Entity: ProductDetail.java
java
@OneToOne(mappedBy = "detail")
private Product product;

ฝั่งนี้เป็น inverse side ของความสัมพันธ์ 1:1 ไม่มี @JoinColumn เพราะไม่ใช่เจ้าของ FK — แค่ประกาศไว้เพื่อจะ navigate กลับจาก ProductDetail ไปหา Product ต้นทางได้ (mappedBy = "detail" ชี้กลับไป field detail ใน Product)

Entity: Review.java
java
@ManyToOne
@JoinColumn(name = "product_id")
private Product product;

@ManyToOne: Review เป็นฝั่ง "Many" — รีวิวหลายรายการเชื่อมกับสินค้าหนึ่งชิ้น

@JoinColumn(name = "product_id"): ฝั่งนี้คือ owning side ที่แท้จริงของความสัมพันธ์ 1:N ทั้งหมด เพราะกฎ JPA กำหนดว่า Foreign Key ต้องอยู่ฝั่ง "Many" เสมอ — คอลัมน์ product_id ในตาราง reviews คือ FK จริงที่ชี้ไปยัง products.id

Service: ProductService.java — Constructor Injection
java
@Service
public class ProductService {
private final ProductRepository productRepository;
@Autowired
public ProductService(ProductRepository productRepository) {
this.productRepository = productRepository;
}
...
}

@Service: บอก Spring ให้จัดการ class นี้เป็น Bean ในชั้น Business Logic

Constructor Injection: แทนที่จะใช้ @Autowired บน field ใดๆ (Field Injection) เราประกาศ productRepository เป็น final แล้วรับค่าผ่าน constructor — ข้อดีคือ:

ทำให้ dependency เป็น immutable (final) ป้องกันการเปลี่ยนแปลงโดยไม่ตั้งใจหลังสร้าง object

เขียน unit test ได้ง่ายกว่า เพราะ mock ProductRepository แล้วส่งเข้า constructor ใดๆ ได้เลยโดยไม่ต้องพึ่ง Spring context

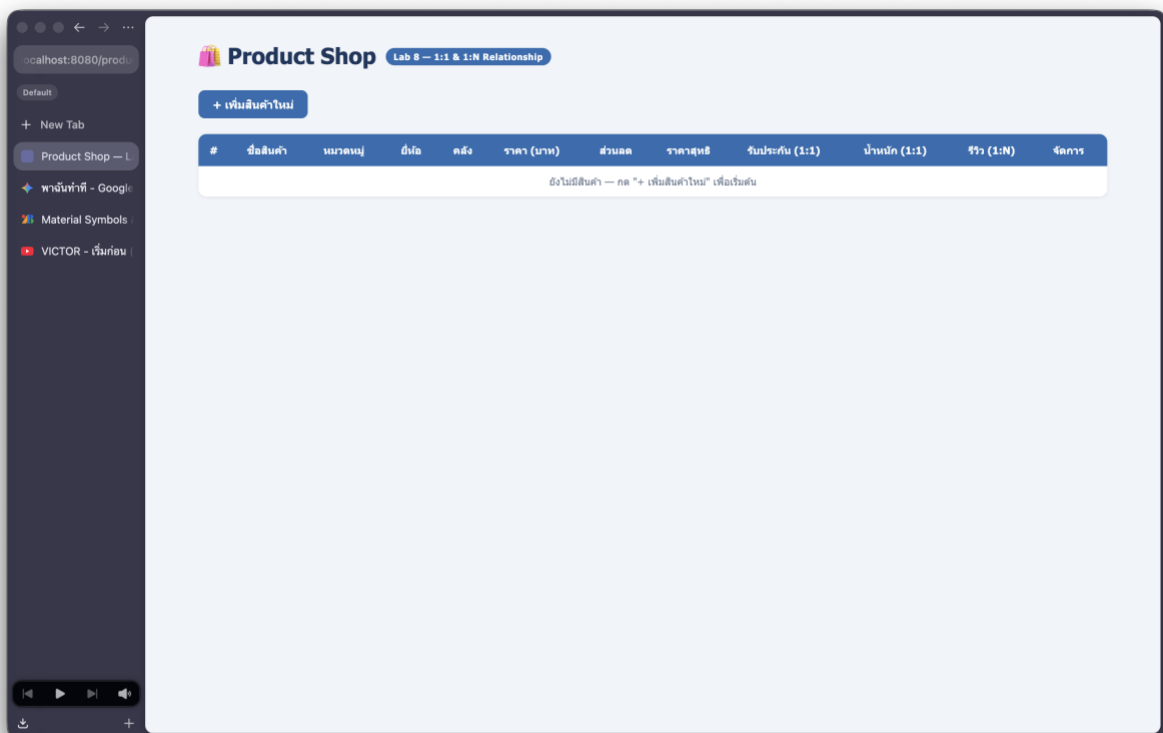
สอดคล้องกับ DIP เพราะ Service ประกาศชัดเจนว่าต้องการ interface อะไรตั้งแต่ตอนสร้าง object ไม่ใช่ปล่อยให้ inject เข้ามาทีหลังแบบซ่อนเร้น

Controller: ProductController.java — Constructor Injection

java
@Controller
@RequestMapping("/products")
public class ProductController {
private final ProductService productService;
@Autowired
public ProductController(ProductService productService) {
this.productService = productService;
}
}

@Controller + @RequestMapping("/products"): กำหนดให้ทุก method ใน class นี้ตอบสนอง HTTP request ที่ path ขึ้นต้นด้วย /products

ใช้ Constructor Injection กับ ProductService ด้วยเหตุผลเดียวกับ Service — Controller ไม่ผูกติดกับ implementation ของ Service โดยตรง ทำให้ตรงตาม SRP (Controller ทำหน้าที่รับ-ส่ง HTTP เท่านั้น ไม่มี business logic ปนอยู่) และ DIP (พึ่งพา abstraction ของ Service layer)



st:8080/products/...

Default

+ New Tab

แก้ไขสินค้า — Lab 8

พาดังที่ - Google

Material Symbols

VICTOR - เริ่มต้น

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 16 Pro (673380418-9 SEC 4)

หมวดหมู่

Phone

ยี่ห้อ

APPLE

จำนวนในคลัง

2

ราคาปกติ (บาท)

15000000000.0

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

รายละเอียดสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

D

ระยะเวลาประกัน

2 Year

น้ำหนัก (กิโลกรัม)

1.90

ขนาด

15 x 7 x 9

ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก

st:8080/products/...

Default

+ New Tab

เพิ่มสินค้า — Lab 8

พาดังที่ - Google

Material Symbols

VICTOR - เริ่มต้น

+ เพิ่มสินค้าใหม่

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380418-9 SEC 4)

หมวดหมู่

Phone

ยี่ห้อ

APPLE

จำนวนในคลัง

200

ราคาปกติ (บาท)

15000

ประเภทส่วนลด

ส่วนลดสมาชิก (10%)

รายละเอียดสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

D

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

1.61

ขนาด (กว้าง x ยาว x สูง)

15 x 7 x 9

ประเทศที่ผลิต

China

รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เห็นรีวิวแรกพร้อมกับสินค้า (สามารถเห็นรีวิวเพิ่มเติมภายหลังได้)

localhost:8080/prod...

Default

+ New Tab

Product Shop — L

พาดีนต้าที่ - Google

Material Symbols

VICTOR - เริ่มต้น

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	จำนวน (1:1)	น้ำหนัก (1:1)	จำนวน (1:N)	จัดการ
1	iPhone 15 Pro (673380418-9 SEC 4)	Phone	APPLE	200	15000.00	MEMBER	13500.00	1 Year	1.61	0 ไร่	แก้ไขลบ

localhost:8080/products/...

Default

+ New Tab

แก้ไขสินค้า - Lab

พาณิชย์ - Google

Material Symbols

VICTOR - เริ่มเรียน

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 16 Pro (673380418-9 SEC 4)

หมวดหมู่

Phone

ยี่ห้อ

APPLE

จำนวนในคลัง

2

ราคาปกติ (บาท)

15000000000.0

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

D

ระยะเวลาประกัน

2 Year

น้ำหนัก (กิโลกรัม)

1.90

ขนาด

15 x 7 x 9

ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก

localhost:8080/prod...

Default

+ New Tab

Product Shop - L

พาณิชย์ - Google

Material Symbols

VICTOR - เริ่มเรียน

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	ราคา (1:N)	จัดการ
1	iPhone 16 Pro (673380418-9 SEC 4)	Phone	APPLE	2	15000000000.00	SEASONAL	12000000000.00	2 Year	1.9	0 ไร่	แก้ไขลบ

